Algebraic Data Types with Sealed Records

Season: 6 · FP Foundations · Category: Functional · Level: Intermediate

Intent

- Sum and product types with exhaustive switch.

Problem

- Without this pattern, code tends toward: New things up with conditionals and duplicated construction.

Real-world Analogy

- Payment = Card | Cash | Wire.

When to Use

- Pure functional style in Java 25 + Zazr.
- The design force named in Intent shows up more than once.

When Not to Use

- Throwing exceptions everywhere is 'fine' for a script.
- A simpler language feature (record, sealed switch, function value) already solves it.

Pattern Structure

- See mermaid diagram in explanation.md

Classic Implementation

```java
package com.buildwithwaqar.patterns.functional.adt;

/** Algebraic data type via sealed records + pattern matching. */
public sealed interface Payment permits CardPayment, CashPayment, WirePayment {
    int amountCents();

    static String describe(Payment p) {
        return switch (p) {
            case CardPayment(var cents, var last4) -> "card ****" + last4 + " " + cents;
            case CashPayment(var cents) -> "cash " + cents;
            case WirePayment(var cents, var iban) -> "wire " + iban + " " + cents;
        };
    }
}

record CardPayment(int amountCents, String last4) implements Payment {}
record CashPayment(int amountCents) implements Payment {}
record WirePayment(int amountCents, String iban) implements Payment {}
```

Modern Java 25 / Functional

- Primary examples use Zazr; VAVR shown in comparison episode.

Worked Example

```
See the JUnit test beside the implementation.
Run: mvn -Dtest=*Test test
Focus package matching: functional/adt/Payment
```

Pitfalls

- Zazr rejects null inside Option/collections.
- Keep interpreters at the edge.

Interview Q&A

- Q: Interview: what problem does Algebraic Data Types with Sealed Records solve? A: Sum and product types with exhaustive switch.
- Q: How would you test this? A: Prefer injecting interfaces or interpreters; keep demos deterministic (fixed clocks, canned inputs).

Key Takeaways

- Sum and product types with exhaustive switch.
- Prefer Java 25 sealed types / records / virtual threads where they simplify the classic shape.
- Ship working tests with every episode.

Video Script Highlights

- [0:00] Hook — Open with the analogy: Payment = Card | Cash | Wire. [0:40] Intent — State the design force in one sentence: Sum and produc...